



Deep Reinforcement Learning-based scheduling for optimizing system load and response time in edge and fog computing environments

Zhiyu Wang^{a,*}, Mohammad Goudarzi^b, Mingming Gong^c, Rajkumar Buyya^a

^a Cloud Computing and Distributed Systems (CLOUDS) Laboratory, The University of Melbourne, Melbourne, Australia

^b School of Computer Science and Engineering, The University of New South Wales (UNSW), Sydney, Australia

^c School of Mathematics and Statistics, The University of Melbourne, Melbourne, Australia

ARTICLE INFO

Keywords:

Edge computing
Fog computing
Machine learning
Deep reinforcement learning
Internet of Things

ABSTRACT

Edge/fog computing, as a distributed computing paradigm, satisfies the low-latency requirements of ever-increasing number of IoT applications and has become the mainstream computing paradigm behind IoT applications. However, because large number of IoT applications require execution on the edge/fog resources, the servers may be overloaded. Hence, it may disrupt the edge/fog servers and also negatively affect IoT applications' response time. Moreover, many IoT applications are composed of dependent components incurring extra constraints for their execution. Besides, edge/fog computing environments and IoT applications are inherently dynamic and stochastic. Thus, efficient and adaptive scheduling of IoT applications in heterogeneous edge/fog computing environments is of paramount importance. However, limited computational resources on edge/fog servers imposes an extra burden for applying optimal but computationally demanding techniques. To overcome these challenges, we propose a Deep Reinforcement Learning-based IoT application Scheduling algorithm, called DRLIS to adaptively and efficiently optimize the response time of heterogeneous IoT applications and balance the load of the edge/fog servers. We implemented DRLIS as a practical scheduler in the FogBus2 function-as-a-service framework for creating an edge–fog–cloud integrated serverless computing environment. Results obtained from extensive experiments show that DRLIS significantly reduces the execution cost of IoT applications by up to 55%, 37%, and 50% in terms of load balancing, response time, and weighted cost, respectively, compared with metaheuristic algorithms and other reinforcement learning techniques.

1. Introduction

The past few years have witnessed the rapid rise of the Internet of Things (IoT) industry, enabling the connection of people to things and things to things, and facilitating the digitization of the physical world [1]. Meanwhile, with the explosive growth of IoT devices and various applications, the expectation for stability and low latency is higher than ever [2]. As the main enabler of IoT, cloud computing stores and processes data and information generated by IoT devices. Leveraging powerful computing capabilities and advanced storage technologies, cloud computing ensures the security and reliability of stored information. However, servers in the cloud computing paradigm are usually located at a long physical distance from IoT devices, and the high latency caused by long distances cannot efficiently satisfy real-time IoT applications. Prompted by these issues, edge and fog computing have emerged as popular computing paradigms in the IoT context. Although some researchers use the terms edge computing and fog computing interchangeably, we clearly define them in this paper. We

consider the case that use “only” edge resources for real-time IoT applications as edge computing, and the case that use edge and whenever necessary also utilizes cloud resources (along with edge resources in a seamless manner) as fog computing. Edge computing as a decentralized computing architecture brings processing, storage, and intelligent control to the vicinity of IoT devices [3]. This flexible architecture extends cloud computing services to the edge of the network. In contrast, the fog computing paradigm inherits the advantages of both cloud and edge computing [4], which not only provides powerful computational capabilities but also reduces the need to transfer data to the cloud for processing, analysis, and storage, thus reducing the inter-network distance. In the real world, edge and fog computing provide strong support for innovation and development in various fields. For example, in the field of smart healthcare, deploying edge computing nodes on wearable devices and medical devices can monitor patients' physiological parameters in real time and transmit the data to the cloud for analysis and diagnosis, realizing telemedicine and personalized medicine [5]; in the

* Corresponding author.

E-mail addresses: zhiyuwang1@student.unimelb.edu.au (Z. Wang), m.goudarzi@unsw.edu.au (M. Goudarzi), mingming.gong@unimelb.edu.au (M. Gong), rbuyya@unimelb.edu.au (R. Buyya).

<https://doi.org/10.1016/j.future.2023.10.012>

Received 28 June 2023; Received in revised form 14 August 2023; Accepted 20 October 2023

Available online 28 October 2023

0167-739X/© 2023 The Author(s). Published by Elsevier B.V. This is an open access article under the CC BY license (<http://creativecommons.org/licenses/by/4.0/>).

field of autonomous driving, deploying edge computing nodes on self-driving vehicles can perform real-time sensing and decision processing, enabling shorter response time and improving driving safety [6].

However, the massive growth in the number of IoT applications and servers in fog computing environments also creates new challenges. Firstly, the execution time is expected to be minimized [7], which means that the applications should be processed by the best (i.e., the most powerful and physically closest) server. Besides, the load should be ideally balanced and distributed to run on multiple operating units. For example, by distributing requests across multiple servers in a seamless manner (as in serverless computing environments), load balancing can avoid overloading individual servers and ensure that each server handles a moderate load. This improves response times, overall system performance, and throughput, and also helps servers run more consistently. Therefore, improving the load balancing level of servers (i.e., lowering the variance of server resource utilization) while reducing the response time becomes an important but challenging problem for scheduling IoT applications on servers in edge/fog computing environments. Since this is an NP-hard problem, metaheuristic and rule-based solutions can be considered [8,9]. However, these approaches often rely on omniscient knowledge of global information and require the solution proponent to have control over the changes. In the fog computing environment, there is often no regularity in server performance, utilization, and downtime. The number of IoT applications and the corresponding resource requirements are even more nearly random. Besides, in reality, Directed Acyclic Graphs (DAGs) are often used to model IoT applications [10], where nodes represent tasks and edges represent data communication between dependent tasks. The dependency among tasks introduces higher complexity in scheduling applications. Therefore, metaheuristic and rule-based solutions cannot efficiently cope with the IoT application scheduling problem in fog computing environments.

Deep Reinforcement Learning (DRL) is the product of combining deep learning with reinforcement learning, integrating the powerful understanding of deep learning on perceptual problems with the decision-making capabilities of reinforcement learning. In deep reinforcement learning, the agent continuously interacts with the environment, recording a large number of empirical trajectories (i.e., sequences of states, actions, and rewards), which are used in the training phase to learn optimal policies. In contrast to metaheuristic algorithms, agents in deep reinforcement learning are able to autonomously sense and respond to changes in the environment, which allows deep reinforcement learning to solve complex problems in realistic scenarios. However, due to the limited computational resources of devices in fog computing environments [11], the computational requirements of complex Deep Neural Networks (DNNs) are often not supported [12]. Therefore, how to balance implementation simplicity, sample complexity, and solution performance becomes a key research problem in applying deep reinforcement learning to fog computing environments to cope with complex situations.

To address the above challenges, we propose a Deep Reinforcement Learning-based IoT application Scheduling algorithm (DRLIS), which employs Proximal Policy Optimization (PPO) [13] technique for solving the IoT applications scheduling problem in fog computing environments. DRLIS can effectively optimize the load balancing cost of the servers, the response time cost of the IoT applications, and their weighted cost. Besides, by using clipped surrogate objective to limit the magnitude of policy updates in each iteration and being able to perform multiple iterations of updates in the sampled data, the convergence speed of the algorithm is improved. Moreover, considering the limited computational resources and the optimization objective under study, we design efficient reward functions. The main contributions of this paper are:

- We propose a weighted cost model regarding DAG-based IoT applications' scheduling in fog computing environments to improve the load balancing level of the servers while minimizing

the response time of the application. In addition, we adapt this weighted cost model to make it applicable to DRL algorithms.

- We propose a DRL-based algorithm (DRLIS) to solve the defined weighted cost optimization problem in dynamic and stochastic fog computing environments. When the computing environment changes (e.g., requests from different IoT applications, server computing resources, the number of servers), it can adaptively update the scheduling policy with a fast convergence speed.
- Based on DRLIS, we implement a practical scheduler in the FogBus2 function-as-a-service framework¹ [14] for handling scheduling requests of IoT applications in heterogeneous fog and edge computing environments. We also extend the functionality of the FogBus2 framework to make different DRL techniques applicable to it.
- We conduct practical experiments and use real IoT applications with heterogeneous tasks and resource demands to evaluate the performance of DRLIS in real system setup. By comparing with common metaheuristics (Non-dominated Sorting Genetic Algorithm 2 (NSGA2) [16], Non-dominated Sorting Genetic Algorithm 3 (NSGA3) [17]) and other reinforcement learning algorithms (Q-Learning [18]), we demonstrate the superiority of DRLIS in terms of convergence speed, optimization cost, and scheduling time.

The rest of the paper is organized as follows. Section 2 discusses related work and Section 3 presents the system model and problem formulation. The Deep Reinforcement Learning model for IoT applications in edge and fog computing environments is presented in Section 4. DRLIS is discussed in Section 5. Section 6 evaluates the performance of DRLIS and compares it with other counterparts. Finally, Section 7 concludes the paper and states future work.

2. Related work

In this section, we review the literature on scheduling IoT applications in edge and fog computing environments. The related works are divided into metaheuristic and reinforcement learning categories.

2.1. Metaheuristic

In the dependent category, Liu et al. [19] adopted a Markov Decision Process (MDP) approach to achieving shorter average task execution latency in edge computing environments. They proposed an efficient one-dimensional search algorithm to find the optimal task scheduling policy. However, this work cannot adapt to changes in the computing environment and is difficult to extend to solve complex weighted cost optimization problems in heterogeneous fog computing environments. Wu et al. [20] modeled the task scheduling problem in edge and fog computing environments as a DAG and used an estimation of distribution algorithm (EDA) and a partitioning operator to partition the graph in order to queue tasks and assign appropriate servers. However, they did not practically implement and test their work. Sun et al. [21] improved the NSGA2 algorithm and designed a resource scheduling scheme among fog nodes in the same fog cluster, taking into account the diversity of different devices. This work aims to reduce the service latency and improve the stability of task execution. Although capable of handling weighted cost optimization problems, this work only considers scheduling problems in the same computing environment. Hoseiny et al. [22] proposed a Genetic Algorithm (GA)-based technique for minimizing the total computation time and energy consumption of task scheduling in a heterogeneous fog cloud computing environment. By introducing features for tasks, the technique can find a more suitable computing environment for each task. However, it does not consider the dependencies of different tasks in the application, and due to the

¹ Please refer to [14,15] for detailed description of the FogBus2 framework.

use of metaheuristic algorithms, scheduling rules need to be manually set, which cannot adapt to changing computing environments. Ali et al. [23] proposed an NSGA2-based technique for minimizing the total computation time and system cost of task scheduling in heterogeneous fog cloud computing environments. Their work formulates the task scheduling problem as an optimization problem in order to dynamically allocate appropriate resources for predefined tasks. Similarly, due to the limitations of metaheuristic algorithms, this work requires the assumption that the technique has some knowledge of the submitted tasks to develop the scheduling policy and thus cannot cope with dynamic and complex scenarios.

2.2. Reinforcement learning

In the dependent category, Shahidani et al. [24] proposed a Q-learning-based algorithm to reduce task execution latency and balance the load in a fog cloud computing environment. However, this work does not consider the inter-task dependencies and the heterogeneity of fog and cloud computing environments. Baek et al. [25] adapted the Q-learning algorithm and proposed an approach that aims at improving load balancing in fog computing environments. This work considers the heterogeneity of nodes in fog computing environments but still assumes that the tasks within the application are independent of each other. Jie et al. [26] proposed a Deep Q-Network (DQN)-based approach to minimize the total latency of task processing in edge computing environments. This work formulates task scheduling as a Markov Decision Process while considering the heterogeneity of IoT applications. However, this work only considers the scheduling problem in edge computing environments and investigates only one optimization objective. Xiong et al. [27] adapted the DQN algorithm and proposed a resource allocation strategy for IoT edge computing systems. This work aims at minimizing the average job completion time but does not take into account more complex functions with multiple optimization objectives. Wang et al. [28] focus on edge computing environments and propose a deep reinforcement learning-based resource allocation (DRLRA) scheme based on DQN. This work targets to reduce the average service time and balance the resource usage within the edge computing environment. However, the work does not consider the resources in fog computing environment, and the technique is not practically implemented and tested. Huang et al. [29] adopted a DQN-based approach to address the resource allocation problem in the edge computing environment. This work investigated minimizing the weighted cost, including the total energy consumption and the latency to complete the task. However, it does not consider the heterogeneity of servers in fog computing environments and assumes that the tasks are independent. Chen et al. [30] proposed an approach based on double DQN to balance task execution time and energy consumption in edge computing environments. Similarly, this work is only applicable to the edge environment and does not consider the dependencies between tasks. Zheng et al. [31] proposed a Soft Actor–Critic (SAC)-based algorithm to minimize the task completion time in an edge computing environment. This work focuses on the latency problem and the experiments are simulation-based. Zhao et al. [32] proposed a Twin Delayed DDPG (TD3)-based DRL algorithm. The goal of this work is to minimize the latency and energy consumption, but inter-task dependencies are not considered and the results are also simulation-based. Liao et al. [33] used Deep Deterministic Policy Gradient (DDPG) and Double Deep Q-Network (DQN) algorithms to model computation in an edge environment. This work aims to reduce energy consumption and latency but does not consider the fog environment and the heterogeneity of devices. Sethi et al. [34] proposed a DQN-based algorithm to optimize energy consumption and load balancing of fog servers. Similarly, this work is simulation-based and does not consider the dependencies between tasks.

Table 1 presents the comparison of the related work with our proposed algorithm, in terms of application properties, architecture

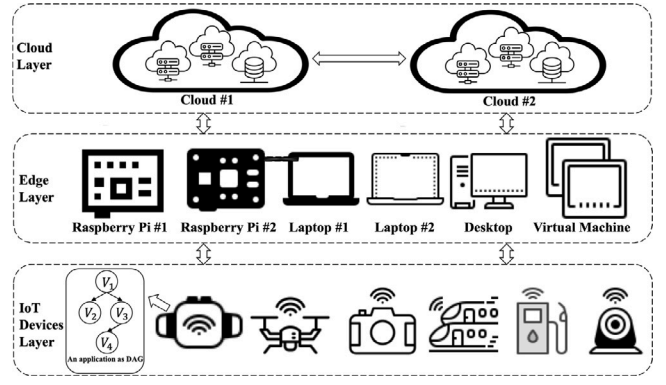


Fig. 1. A view of the IoT system in fog computing.

properties, algorithm properties, and evaluation. In the application properties section, the number of tasks included in the IoT application, and the dependencies between tasks are studied. In the architectural properties section, three aspects are studied including the IoT device layer, the edge/fog layer, and the multi-cloud layer. For the IoT device layer, the application type and request type are identified. The real application section indicates that the work either deploys actual IoT applications, adopts simulated applications, or uses random data. The heterogeneous request type represents work considering that different IoT devices have different numbers of requests and different requirements. For the edge/fog layer, the computing environment and the heterogeneity of deployed servers are investigated. Besides, the multi-cloud layer studies whether the work considers the scenario of different cloud service providers with heterogeneity. In the algorithm properties section, we investigate the main technique on which each work is based and the corresponding optimization objectives. The evaluation section identifies whether the work is based on simulation or practical experiments. Recent works that we reviewed (e.g., [31–37]) have often used reinforcement learning approaches to deal with workload scheduling problems. This is because reinforcement learning can learn by interacting with the environment and continuously optimizing the policy through feedback signals (e.g., reward or penalty). This learning ability gives reinforcement learning an advantage when facing complex, dynamic environments [38], whereas metaheuristic techniques require manual adaptation and guidance.

3. System model and problem formulation

In this section, we first introduce the topology of the IoT systems in the edge and fog computing environment. Then, we discuss the problem formulation. The key notations are listed in Table 2.

3.1. System model

Fig. 1 represents a layered view of the IoT Systems in the fog computing environment. Consider $S = \{S_l | 1 \leq l \leq |S|\}$ as a collection of $|S|$ applications, where each application contains one or more tasks, denoted as $S_l = \{S_{li} | 1 \leq i \leq |S_l|\}$. The DAG $G = (V, E)$ is used to model an IoT application, as depicted in Fig. 2. A vertex $v_i = S_{li}$ denotes a certain task of the application, and an edge $e_{i,j}$ denotes the data flow between tasks v_i and v_j , so some tasks must be executed after predecessor tasks are completed. $CP(S_l)$ represents the critical path (i.e., the path with the highest cost) of the DAG, marked in red in the figure.

A set containing $|N|$ servers is used to process application set S , denoted as $N = \{n_k | 1 \leq k \leq |N|\}$. To reflect the heterogeneity of the servers, for each server n_k , $n_k^{cpu,ut}$ represents its CPU utilization (%), n_k^{freq} represents its CPU frequency (MHz), $n_k^{ram,ut}$ represents its RAM

Table 1

A qualitative comparison of related works with ours.

Works	Application properties		Architectural properties				Algorithm properties						Evaluation
	Task number	Dependency	IoT device layer		Edge/Fog layer		Multi-Cloud layer	Main technique	Optimization objectives				
			Real applications	Request type	Computing environments	Heterogeneity			Time	Load balancing	Weighted		
[19]	Single	Independent	○	Homogeneous	Edge	Homogeneous	×	Metaheuristic Algorithms	MDP	✓	×	×	Simulation
[21]	Multiple		●	Homogeneous	Edge and Fog	Heterogeneous	×		NSGA2	✓	×	✓	Simulation
[22]	Single		○	Homogeneous	Edge and Fog	Heterogeneous	×		GA	✓	×	×	Simulation
[23]	Single		○	Homogeneous	Edge and Fog	Heterogeneous	×		NSGA2	✓	×	✓	Simulation
[20]	Multiple	Dependent	●	Homogeneous	Edge and Fog	Heterogeneous	×	EDA	✓	×	✓	Simulation	
[25]	Single	Independent	○	Homogeneous	Edge and Fog	Heterogeneous	×	Reinforcement Learning Techniques	Q-Learning	×	✓	×	Simulation
[24]	Single		○	Homogeneous	Edge and Fog	Homogeneous	×		Q-Learning	✓	✓	✓	Simulation
[26]	Single		●	Homogeneous	Edge	Homogeneous	×		DQN	✓	×	×	Simulation
[27]	Multiple		●	Homogeneous	Edge	Homogeneous	×		DQN	✓	×	×	Simulation
[29]	Multiple		●	Heterogeneous	Edge	Homogeneous	×		DQN	✓	×	✓	Simulation
[28]	Single		●	Homogeneous	Edge	Homogeneous	×		DQN	✓	✓	✓	Simulation
[30]	Single		●	Heterogeneous	Edge	Homogeneous	×		Double DQN	✓	×	✓	Simulation
[31]	Single		●	Homogeneous	Edge	Homogeneous	×		SAC	✓	×	×	Simulation
[32]	Single		●	Homogeneous	Edge and Fog	Homogeneous	×		TD3	✓	×	✓	Simulation
[35]	Single		○	Homogeneous	Edge	Homogeneous	×		DQN	×	✓	×	Simulation
[33]	Single		●	Homogeneous	Edge	Homogeneous	×		DDPG and DQN	✓	×	✓	Simulation
[36]	Single	Independent	○	Homogeneous	Edge	Homogeneous	×	DDPG	✓	×	✓	Simulation	
[34]	Single		●	Homogeneous	Edge and Fog	Homogeneous	×	DQN	×	✓	✓	Simulation	
[37]	Multiple	Dependent	●	Heterogeneous	Edge	Heterogeneous	×	GA and DQN	✓	×	×	Simulation	
DRLIS	Multiple		●	Heterogeneous	Edge and Fog	Heterogeneous	✓	PPO	✓	✓	✓	Practical	

●: Real IoT Application and Deployment, ●: Simulated IoT Application, ○: Random.

Table 2

List of key notations.

Variable	Description	Variable	Description
S	The application set	$\psi_{x_{S_{l_i}}}^{ram}$	The variance of RAM utilization of the server set after the scheduling configuration $x_{S_{l_i}}$
S_l	One application (one task set)	$\Psi(\chi_l)$	The load balancing model after the scheduling configuration χ_l
S_{l_i}	One task	$\Psi(\chi)$	The load balancing model after the scheduling configuration χ
N	The server set	$\omega_{x_{S_{l_i}}}$	The total execution time (ms) for task S_{l_i} based on the scheduling configuration $x_{S_{l_i}}$
$x_{S_{l_i}}$	The scheduling configuration of task S_{l_i}	$\omega_{x_{S_{l_i}}}^{rrt}$	The ready time (ms) for task S_{l_i} based on the scheduling configuration $x_{S_{l_i}}$
χ_l	The scheduling configuration of application S_l	ω_{n_j, n_k}^{rrt}	The time (ms) consumed for required data by task S_{l_i} to be sent from server n_j to server n_k
χ	The scheduling configuration of applications S	$P(S_{l_i})$	The parent tasks set of task $x_{S_{l_i}}$
$n_k^{cpu_uti}$	The CPU utilization (%) of server n_k	$PS(S_{l_i})$	The server set to which the dependency tasks of task $x_{S_{l_i}}$ are assigned
n_k^{freq}	The CPU frequency (MHz) of server n_k	$\omega_{n_j, n_k}^{trans}$	The transmission time (ms) between server n_j and server n_k
$n_k^{ram_uti}$	The RAM utilization (%) of server n_k	ω_{n_j, n_k}^{prop}	The propagation time (ms) between server n_j and server n_k
$n_k^{ram_size}$	The RAM size (GB) of server n_k	p_{n_j, n_k}	The packet size (MB) from server n_j to server n_k for task S_{l_i}
N^{cpu_uti}	The CPU utilization (%) of each server in server set N , denoted as a set	b_{n_j, n_k}	The data rate (bit/s) between server n_j and server n_k
N^{ram_uti}	The RAM utilization (%) of each server in server set N , denoted as a set	$CP(S_{l_i})$	Equals to 1 if S_{l_i} is on the critical path of application S_l , otherwise 0
$S_{l_i}^{ram}$	The minimum RAM required for executing task S_{l_i}	$\omega_{x_{S_{l_i}}}^{proc}$	The processing time (ms) for task S_{l_i} based on the scheduling configuration $x_{S_{l_i}}$
$\psi_{x_{S_{l_i}}}$	The load balancing model after the scheduling configuration $x_{S_{l_i}}$	$\Omega(\chi_l)$	The total execution time (ms) for application S_l based on the scheduling configuration χ_l
$\psi_{x_{S_{l_i}}}^{cpu}$	The variance of CPU utilization of the server set after the scheduling configuration $x_{S_{l_i}}$	$\Omega(\chi)$	The total execution time (ms) for the application set S based on the scheduling configuration χ

utilization (%), and $n_k^{ram_size}$ represents its RAM size (GB). Moreover, $PS(S_{l_i})$ represents the server set to which the parent tasks of task S_{l_i} are assigned, and $\omega_{n_j, n_k}^{trans}$, ω_{n_j, n_k}^{prop} , p_{n_j, n_k} , and b_{n_j, n_k} denote the transmission time (ms), the propagation time (ms), the packet size (MB), and the data rate (bit/s) between server n_j and server n_k , respectively.

3.2. Problem formulation

Since an application contains one/multiple tasks, it may be executed on different servers. With a set of servers N , the scheduling

configuration $x_{S_{l_i}}$ of a task S_{l_i} is defined as:

$$x_{S_{l_i}} = \{n_k\}, \quad (1)$$

where k shows the server's index. Accordingly, the scheduling configuration χ_l of an application S_l is equal to the set of the scheduling configuration of the tasks it contains, defined as:

$$\chi_l = \{x_{S_{l_i}} | S_{l_i} \in S_l, 1 \leq i \leq |S_l|\}. \quad (2)$$

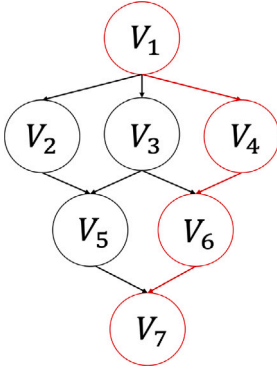


Fig. 2. Sample IoT application with the critical path in red color.

The scheduling configuration χ of the application set S is equal to the set of scheduling configuration per application:

$$\chi = \{\chi_l | 1 \leq l \leq |S|\}. \quad (3)$$

In addition, we consider that for a given application, the execution model of tasks can be hybrid (i.e., sequential and/or parallel). That is, children tasks have some dependencies on the parent tasks that need to be executed after their completion, and we use $P(S_{l_i})$ to represent the parent task set of task S_{l_i} [39]. While tasks that do not depend on each other can be executed in parallel, and we use $CP(S_{l_i})$ to indicate that if a task S_{l_i} is located on a critical path of application S_l .

3.2.1. Load balancing model

The load balancing model is used to measure the resource balancing level of the server set N during the processing of the application set S . Regarding the server resource, both CPU and RAM are considered. For task S_{l_i} , the load balancing model $\psi_{x_{S_{l_i}}}$ is defined as:

$$\psi_{x_{S_{l_i}}} = a_1 \psi_{x_{S_{l_i}}}^{cpu} + a_2 \psi_{x_{S_{l_i}}}^{ram}, \quad (4)$$

where $\psi_{x_{S_{l_i}}}^{cpu}$ and $\psi_{x_{S_{l_i}}}^{ram}$ represent the CPU and RAM models, and a_1 and a_2 are the control parameters by which the weighted load balancing model can be tuned. They satisfy:

$$a_1 + a_2 = 1, 0 \leq a_1, a_2 \leq 1. \quad (5)$$

CPU model $\psi_{x_{S_{l_i}}}^{cpu}$ and RAM model $\psi_{x_{S_{l_i}}}^{ram}$ are defined as the variance of CPU and RAM utilization of the server set N after the scheduling configuration $x_{S_{l_i}}$:

$$\psi_{x_{S_{l_i}}}^{cpu} = \text{Var}[N^{cpu_uti}], \quad (6)$$

$$\psi_{x_{S_{l_i}}}^{ram} = \text{Var}[N^{ram_uti}], \quad (7)$$

where

$$x_{S_{l_i}} = \{n_k\}. \quad (1)$$

Correspondingly, for application S_l , the load balancing model $\Psi(\chi_l)$ is defined as the sum of the load balancing models for each task processed by server set N :

$$\Psi(\chi_l) = \sum_{i=1}^{|S_l|} \psi_{x_{S_{l_i}}}. \quad (8)$$

Our main goal is to find the best-possible scheduling configuration for the application set S such that the variance of the overall CPU and RAM utilization of the server set N during the processing of the

application set S can be minimized. Therefore, for the application set S , the load balancing model $\Psi(\chi)$ is defined as:

$$\Psi(\chi) = \sum_{l=1}^{|S|} \Psi(\chi_l) = \sum_{l=1}^{|S|} \sum_{i=1}^{|S_l|} \psi_{x_{S_{l_i}}}. \quad (9)$$

3.2.2. Response time model

We consider the response time model $\omega_{x_{S_{l_i}}}$ for the task S_{l_i} consisting of two components, the task ready time model $\omega_{x_{S_{l_i}}}^{trt}$ and the processing model $\omega_{x_{S_{l_i}}}^{proc}$:

$$\omega_{x_{S_{l_i}}} = \omega_{x_{S_{l_i}}}^{trt} + \omega_{x_{S_{l_i}}}^{proc}. \quad (10)$$

The task ready time model $\omega_{x_{S_{l_i}}}^{trt}$ represents the maximum time for the data required by the task S_{l_i} to arrive at the server to which it is assigned, defined as:

$$\omega_{x_{S_{l_i}}}^{trt} = \max \omega_{n_j, n_k}^{trt}, \quad \forall n_j \in PS(S_{l_i}), \quad (11)$$

where ω_{n_j, n_k}^{trt} denotes the time consumed for required data by task S_{l_i} sent from server n_j to server n_k , and n_k is the server where the task S_{l_i} will be executed based on scheduling configuration $x_{S_{l_i}}$, and n_j represents the server where the parent task of task S_{l_i} is executed. Therefore, ω_{n_j, n_k}^{trt} depends on the transmission time $\omega_{n_j, n_k}^{trans}$ and the propagation time ω_{n_j, n_k}^{prop} for task S_{l_i} between server n_j and server n_k :

$$\omega_{n_j, n_k}^{trt} = \begin{cases} \omega_{n_j, n_k}^{trans} + \omega_{n_j, n_k}^{prop} & n_j \neq n_k, \\ 0 & n_j = n_k. \end{cases} \quad (12)$$

And the transmission time $\omega_{n_j, n_k}^{trans}$ can be calculated as:

$$\omega_{n_j, n_k}^{trans} = \frac{p_{n_j, n_k}}{b_{n_j, n_k}}, \quad (13)$$

where p_{n_j, n_k} represents the packet size from server n_j to server n_k for task S_{l_i} , and b_{n_j, n_k} represents the current bandwidth between server n_j and server n_k when the data for task S_{l_i} is transmitted.

The processing model $\omega_{x_{S_{l_i}}}^{proc}$ is defined as the time it takes for assigned server n_k to process the task S_{l_i} based on scheduling configuration $x_{S_{l_i}}$, and can be calculated as:

$$\omega_{x_{S_{l_i}}}^{proc} = \frac{S_{l_i}^{size}}{n_k^{freq}}, \quad (14)$$

where $S_{l_i}^{size}$ represents the required CPU cycles for task S_{l_i} and n_k^{freq} represents the CPU frequency of server n_k (for multi-core CPUs, the average frequency is considered).

Accordingly, the response time model $\Omega(\chi_l)$ for application S_l is defined as:

$$\Omega(\chi_l) = \sum_{i=1}^{|S_l|} (\omega_{x_{S_{l_i}}} \times CP(S_{l_i})), \quad (15)$$

where $CP(S_{l_i})$ equals to 1 if task S_{l_i} is on the critical path of application S_l , otherwise 0.

The main goal for the response time model $\Omega(\chi)$ is to find the best-possible scheduling configuration for the application set S such that the total time for the server set N processing them can be minimized. Therefore, for the application set S , the response time model $\Omega(\chi)$ is defined as:

$$\Omega(\chi) = \sum_{l=1}^{|S|} \Omega(\chi_l) = \sum_{l=1}^{|S|} \sum_{i=1}^{|S_l|} (\omega_{x_{S_{l_i}}} \times CP(S_{l_i})). \quad (16)$$

3.2.3. Weighted cost model

The weighted cost model is defined as the weighted sum of the normalized load balancing and normalized response time models. For

task S_{l_i} :

$$\phi_{x_{S_{l_i}}} = w_1 \frac{\psi_{x_{S_{l_i}}} - \psi^{\min}}{\psi^{\max} - \psi^{\min}} + w_2 \frac{\omega_{x_{S_{l_i}}} - \omega^{\min}}{\omega^{\max} - \omega^{\min}}, \quad (17)$$

where $\psi_{x_{S_{l_i}}}$ and $\omega_{x_{S_{l_i}}}$ are the load balancing model and response time model of task S_{l_i} , and $\psi^{\min}$, $\psi^{\max}$, $\omega^{\min}$, and $\omega^{\max}$ represent the minimum and the maximum value of the load balancing model and response time model, respectively. Moreover, w_1 and w_2 are the control parameters by which the weighted cost model can be tuned. The reason we use the normalized models instead of the original models is that the values of the two models may be in different ranges. For example, the load balancing model may have a value from 0 to 1, while the response time model may have a value from 0 to 100. We need to normalize them so that the model values are in the same range.

Accordingly, the weighted cost model for application S_j is defined as:

$$\Phi(\chi_l) = w_1 \times \text{Norm}(\Psi(\chi_l)) + w_2 \times \text{Norm}(\Omega(\chi_l)), \quad (18)$$

where $\Psi(\chi_l)$ and $\Omega(\chi_l)$ are obtained from Eqs. (8) and (15), and Norm represents the normalization. The weighted cost model for the application set S is defined as:

$$\Phi(\chi) = w_1 \times \text{Norm}(\Psi(\chi)) + w_2 \times \text{Norm}(\Omega(\chi)), \quad (19)$$

where $\Psi(\chi)$ and $\Omega(\chi)$ are obtained from Eqs. (9) and (16).

Therefore, the weighted cost optimization problem of IoT applications can be formulated as:

$$\min \Phi(\chi) \quad (20)$$

$$\text{s.t. } C1 : \text{Size}(x_{S_{l_i}}) = 1, \forall x_{S_{l_i}} \in \chi_l \quad (21)$$

$$C2 : 0 \leq n_k^{\text{ram}, \text{ut}}, n_k^{\text{cpu}, \text{ut}} \leq 1, \forall n_k \in N \quad (22)$$

$$C3 : n_k^{\text{freq}}, n_k^{\text{ram}, \text{size}} \geq 0, \forall n_k \in N \quad (23)$$

$$C4 : S_{l_i}^{\text{ram}} < n_k^{\text{ram}, \text{size}}, \forall S_{l_i} \in S_l, \forall n_k \in N \quad (24)$$

$$C5 : \Phi(x_{S_{l_j}}) \leq \Phi(x_{S_{l_j}} + x_{S_{l_i}}), \forall S_{l_j} \in P(S_{l_i}) \quad (25)$$

$$C6 : w_1 + w_2 = 1, 0 \leq w_1, w_2 \leq 1 \quad (26)$$

where $C1$ states that any task can only be assigned to one server for processing. $C2$ states that for any server, the CPU utilization and RAM utilization are between 0 and 1. Besides, $C3$ states that the CPU frequency and the RAM size of any server are larger than 0. Moreover, $C4$ denotes that any server should have sufficient RAM resources to process any task. Also, $C5$ denotes that any task can only be processed after its parent tasks have been processed, and thus the cumulative cost is always larger than or equal to the parent task. In addition, $C6$ denotes that the control parameters of the weighted cost model can only take value from 0 to 1, and the sum of them should be equal to 1.

The problem being formulated is presented to be a non-convex optimization problem, because there may be an infinite number of local optima in the set of feasible domains, and usually, the complexity of the algorithm to find the global optimum is exponential (NP-hard) [40]. To cope with such non-convex optimization problems, most work decomposes them into several convex sub-problems and then solves these sub-problems iteratively until the algorithm converges [41]. This type of approach reduces the complexity of the original problem at the expense of accuracy [42]. In addition, such approaches are highly dependent on the current environment and cannot be applied in dynamic environments with complex and continuously changeable parameters and computational resources [42]. To deal with this problem, we propose DRLIS to efficiently handle uncertainties in dynamic environments by learning from interaction with the environment.

4. Deep reinforcement learning model

In reinforcement learning, the autonomous agent first interacts with the surrounding environment through action. Under the action and the environment, the agent generates a new state, while the environment gives an immediate reward. In this cycle, the agent interacts with the environment continuously and thus generates sufficient data. The reinforcement learning algorithm uses the generated data to modify its own action policy, then interacts with the environment to generate new data, and uses the new data to further improve its behavior. Formally, we use Markov Decision Process (MDP) to model the reinforcement learning problem. Specifically, the learning problem can be described by the tuple $\langle \mathbb{S}, \mathbb{A}, \mathbb{P}, \mathbb{R}, \gamma \rangle$, where $\mathbb{S}$ denotes a finite set of states; $\mathbb{A}$ denotes a finite set of actions; $\mathbb{P}$ denotes the state transition probability; $\mathbb{R}$ denotes the reward function; $\gamma \in [0, 1]$ is the discount factor, used to compute the cumulative rewards.

We assume that the time $\mathbb{T}$ of the learning process is divided into multiple time steps t and the agent will interact with the environment at each time step and have multiple states S_t . At a particular time step t , the agent possesses the environment state $S_t = s$, where $s \in \mathbb{S}$. The agent chooses an action $A_t = a$ according to the policy $\pi(a|s)$, where $a \in \mathbb{A}$, and $\pi(a|s) = \text{Pr}[A_t = a | S_t = s]$ is the policy function, which denotes the probability of choosing the action a in state s . After choosing action a , the agent receives a reward $r = \mathbb{R}[S_t = s, A_t = a]$ from the environment based on the reward function $\mathbb{R}$, and it moves to the next state $S_{t+1} = s'$ based on the state transition function $P_{ss'}^a = \mathbb{P}[S_{t+1} = s' | S_t = s, A_t = a]$. The goal of the reinforcement learning agent is to learn a policy π that maximizes the expectation of cumulative discounted reward $\mathbb{E}_\pi[\sum_{t \in \mathbb{T}} \gamma^t r_t]$.

Based on the weighted cost optimization problem of IoT applications in edge and fog computing environments, the state space $\mathbb{S}$, action space $\mathbb{A}$, and reward function $\mathbb{R}$ for the MDP are defined as follows:

- **State space $\mathbb{S}$:** Since the optimization problem is related to tasks and servers, the state of the problem consists of the feature space of the task currently being processed and the state space of the current server set N . Based on the discussion in Section 3, at the time step t , the feature space of the task S_{l_i} includes the task ID, the tasks' predecessors and successors, the application ID to which the task belongs, the number of tasks in the current application, the estimate of the occupied CPU resources for the execution of the task, the task's RAM requirements, the estimate of the task's response time, etc. Formally, the feature space $\mathbb{F}$ for task S_{l_i} at the time step t is defined as follows:

$$\mathbb{F}_t(S_{l_i}) = \{f_i^y(S_{l_i}) | S_{l_i} \in S_t, 0 \leq y \leq |\mathbb{F}|\}, \quad (27)$$

where y represents the index of the feature in the task feature space $\mathbb{F}$, and $|\mathbb{F}|$ represents the number of features. Moreover, at the time step t , the state space of the current server set N includes the number of servers, each server's CPU utilization, CPU frequency, RAM utilization, and RAM size, and the propagation time and bandwidth between different servers, etc. Formally, the state space $\mathbb{G}$ for the server set N at the time step t is defined as:

$$\mathbb{G}_t(N) = \{|N|, g_t^z(n_k), h_t^q(n_j, n_k) | n_j, n_k \in N, 0 \leq z \leq |g|, 0 \leq q \leq |h|\}, \quad (28)$$

where g represents the state type that is related to only one server (i.e., CPU utilization), z represents its index, and $|g|$ represents the length of this type of state; besides, h denotes the state type that is related to two servers (i.e., propagation time), and similarly, q represents its index and $|h|$ represents the length of this type of state. Therefore, the state space $\mathbb{S}$ is defined as:

$$\mathbb{S} = \{S_t = (\mathbb{F}_t(S_{l_i}), \mathbb{G}_t(N)) | S_{l_i} \in S_t, t \in \mathbb{T}\}. \quad (29)$$

- **Action space $\mathbb{A}$:** The goal is to find the best-possible scheduling configuration for the application set S to minimize the objective function Eq. (20). Therefore, at the time step t , the action can be defined as the assignment of the server to the task S_{i_t} :

$$A_t = x_{S_{i_t}} = n_k. \quad (30)$$

Accordingly, the action space $\mathbb{A}$ can be defined as the server set N :

$$\mathbb{A} = N. \quad (31)$$

- **Reward function $\mathbb{R}$:** Since this is a weighted cost optimization problem, we need to define the reward function for each sub-problem. First, as the *penalty*, a very large negative value is introduced if the task cannot be processed on the assigned server for any reason. Also, for the load balancing problem, based on the discussion in Section 3.2.1, the reward function r_t^{lb} is defined as:

$$r_t^{lb} = \begin{cases} \psi_{x_{S_{i_t-1}}} - \psi_{x_{S_{i_t}}} & \text{succed} \\ \text{penalty} & \text{fail,} \end{cases} \quad (32)$$

where $\psi_{x_{S_{i_t-1}}}$ is obtained from Eq. (4). The value output by reward function r_t^{lb} is the difference between the load balancing models of the server set after scheduling the current task and the previous one. If the value of the load balancing model of the server set is reduced after scheduling the current task, the output reward is positive, otherwise it is negative. Beside, for the response time problem, based on the discussion in Section 3.2.2, the reward function r_t^{rt} is defined as:

$$r_t^{rt} = \begin{cases} \omega_{x_{S_{i_t}}}^{mean} - \omega_{x_{S_{i_t}}} & \text{succed} \\ \text{penalty} & \text{fail,} \end{cases} \quad (33)$$

where $\omega_{x_{S_{i_t}}}$ is obtained from Eq. (10), and $\omega_{x_{S_{i_t}}}^{mean}$ represents the average response time for task S_{i_t} . The value output by reward function r_t^{rt} is the difference between the average response time (the current response time is also considered) and the current response time for task S_{i_t} . If the current response time is lower than the average one, the output reward is positive, otherwise it is negative. The reward function r_t for the weighted cost optimization problem is defined as:

$$r_t = \begin{cases} w_1 \times \text{Norm}(r_t^{lb}) + w_2 \times \text{Norm}(r_t^{rt}) & \text{succed} \\ \text{penalty} & \text{fail,} \end{cases} \quad (34)$$

where w_1 and w_2 are the control parameters, and *Norm* represents the normalization process.

Currently, many advanced deep reinforcement learning algorithms (e.g., PPO, TD3, SAC) have been proposed by different researchers. They show excellent performance in different fields. PPO improves convergence and sampling efficiency by adopting importance sampling and proportional clipping [13]. TD3 (Twin Delayed DDPG) introduces a dual Q network and delayed update strategy to effectively solve the overestimation problem in the continuous action space [43]. SAC (Soft Actor-Critic) combines policy optimization and learning of Q-value functions, providing more robust and exploratory policy learning through maximum entropy theory [44]. These algorithms have achieved remarkable results in different tasks and environments. In our research problem, the agent's action and state space is discrete, which hinders the application of TD3, because it is designed for continuous control [45]. In addition, the original SAC only considers the problem of continuous space [44], although there are some works discussing how to apply SAC to discrete space, they usually need to adopt some special tricks and extensions, such as using soft-max or

sample-prune techniques to accommodate discrete actions [46]. Besides, Wang et al. [47] shows that SAC requires more computation time and convergence time than PPO. Whereas our study focuses on edge and fog computing environments, where handling latency sensitivity and variation are important considerations for choosing the appropriate DRL algorithm. We choose PPO as the basis of DRLIS, because PPO is designed to be more easily adaptable to discrete action spaces [48] and we aim for the algorithm to converge quickly and perform well in diverse environments.

5. DRL-based optimization algorithm

Based on the above-mentioned MDP model, we propose DRLIS to achieve weighted cost optimization of IoT applications in edge and fog computing environments. In this section, we introduce the mathematical principle of the PPO algorithm and discuss the proposed DRLIS.

5.1. Preliminaries

The PPO algorithm belongs to the Policy Gradient (PG) algorithm which considers the impact of actions on rewards and adjusts the probability of actions [49]. We use the same notations as in Section 3 to describe the algorithm. We consider the time horizon $\mathbb{T}$ is divided into multiple time steps t , and the agent has a policy π_θ for determining its actions and interactions with the environment. The objective can be expressed as adjusting the parameter θ to maximize the expected cumulative discounted rewards $\mathbb{E}_{\pi_\theta}[\sum_{t \in \mathbb{T}} \gamma_t r_t]$ [13], expressed by the formula:

$$J(\theta) = \mathbb{E}_{\pi_\theta}[\sum_{t \in \mathbb{T}} \gamma_t r_t]. \quad (35)$$

Since this is a maximization problem, the gradient ascent algorithm can be used to find the maximum value:

$$\theta' = \theta + \alpha \nabla_\theta J(\theta). \quad (36)$$

The key is to obtain the gradient of the reward function $J(\theta)$ with respect to θ , which is called the policy gradient. The algorithm for solving reinforcement problems by optimizing the policy gradient is called the policy gradient algorithm. The policy gradient can be presented as,

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta}[\nabla_\theta \log \pi_\theta(a_t | s_t) A_\theta(a_t | s_t)], \quad (37)$$

where $A_\theta(a_t | s_t)$ is the advantage function at time step t , used to evaluate the action a_t at the state s_t . Here, the policy gradient indicates the expectation of $\nabla_\theta \log \pi_\theta(a_t | s_t) A_\theta(a_t | s_t)$, which can be estimated using the empirical average obtained by sampling. However, the PG algorithm is very sensitive to the update step size, and choosing a suitable step size is challenging [50]. Moreover, practice shows that the difference between old and new policies in training is usually large [13].

To address this problem, Trust Region Policy Optimization (TRPO) [51] is proposed. This algorithm introduces importance sampling to evaluate the difference between the old and new policies and restricts the new policy if the importance sampling ratio grows large. Importance sampling refers to replacing the original sampling distribution with a new one to make sampling easier or more efficient. Specifically, TRPO maintains two policies, the first policy $\pi_{\theta_{old}}$ is the current policy to be refined, and the second policy π_θ is used to collect the samples. The optimization problem is defined as follows:

$$\underset{\theta}{\text{maximize}} \quad \mathbb{E}_t[\frac{\pi_\theta(a_t | s_t)}{\pi_{\theta_{old}}(a_t | s_t)} A_t] \quad (38)$$

$$\text{subject to} \quad \mathbb{E}_t[KL[\pi_{\theta_{old}}(\cdot | s_t), \pi_\theta(\cdot | s_t)]] \leq \delta, \quad (39)$$

where KL represents Kullback–Leibler Divergence, used to quantify the difference between two probability distributions [52], and δ represents the restriction of the update between old policy $\pi_{\theta_{old}}$ and new policy

π_θ . After linear approximation of the objective and quadratic approximation of the constraints, the problem can be efficiently approximated using the conjugate gradient algorithm. However, the computation of conjugate gradient makes the implementation of TRPO more complex and inflexible in practice [53,54].

To make this algorithm well applied in practice, the KL-PPO algorithm [13] is proposed. Rather than using the constraint function $\mathbb{E}_t[KL[\pi_{\theta_{old}}(\cdot|s_t), \pi_\theta(\cdot|s_t)]] \leq \delta$, the KL divergence is added as a penalty in the objective function:

$$L^{KLPP}(\theta) = \mathbb{E}_t[r_t(\theta)A_t - \beta KL[\pi_{\theta_{old}}(\cdot|s_t), \pi_\theta(\cdot|s_t)]], \quad (40)$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$ is the ratio of the new policy and the old policy,

obtained in Eq. (38), and the parameter β can be dynamically adjusted during the iterative process according to the KL divergence. If the current KL divergence is larger than the predefined maximum value, indicating that the penalty is not strong enough and the parameter β needs to be increased. Conversely, if the current KL divergence is smaller than the predefined minimum value, the parameter β needs to be reduced.

Moreover, another idea to restrict the difference between old policy $\pi_{\theta_{old}}$ and new policy π_θ is to use clipped surrogate function *clip*. The PPO algorithm using the clip function (CLIP-PPO) removes the KL penalty and the need for adaptive updates to simplify the algorithm. Practice shows CLIP-PPO usually performs better than KL-PPO [13]. Formally, the objective function of CLIP-PPO is defined as follows:

$$L^{CLIP}(\theta) = \mathbb{E}_t[\min(r_t(\theta)A_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)A_t)]. \quad (41)$$

And $\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)$ restrict the ratio $r_t(\theta)$ into $(1 - \epsilon, 1 + \epsilon)$, defined as:

$$\text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon) = \begin{cases} 1 - \epsilon & r_t(\theta) < 1 - \epsilon \\ r_t(\theta) & 1 - \epsilon \leq r_t(\theta) \leq 1 + \epsilon \\ 1 + \epsilon & r_t(\theta) > 1 + \epsilon. \end{cases} \quad (42)$$

By removing the constraint function as discussed in TRPO, both PPO algorithms significantly reduce the computational complexity, while ensuring that the updated policy deviates not too large from the previous one.

5.2. DRLIS: DRL-based IoT application scheduling

Since CLIP-PPO usually outperforms KL-PPO in practice, we choose it as the basis for the optimization algorithm. DRLIS is based on the actor-critic framework, which is a reinforcement learning method combining Policy Gradient and Temporal Differential (TD) learning. As the name implies, this framework consists of two parts, the actor and the critic, and in implementation, they are usually presented as Deep Neural Networks (DNNs). The actor network is used to learn a policy function $\pi_\theta(a|s)$ to maximize the expected cumulative discounted reward $\mathbb{E}_\pi[\sum_{i \in T} \gamma^i r_i]$, while the critic network is used to evaluate the current policy and to guide the next stage of the actor's action. In the learning process, at the time step t , the reinforcement learning agent inputs the current state s_t into the actor network, and the actor network outputs the action a_t to be performed by the agent in the MDP. The agent performs the action a_t , receives the reward r_t from the environment, and moves to the next state s_{t+1} . The critic network receives the states s_t and s_{t+1} as input and estimates their value functions $V_{\pi_\theta}(s_t)$ and $V_{\pi_\theta}(s_{t+1})$. The agent then computes the TD error δ_t for the time step t :

$$\delta_t = r_t + \gamma V_{\pi_\theta}(s_{t+1}) - V_{\pi_\theta}(s_t), \quad (43)$$

where γ denotes the discount factor, as discussed in Section 3, and the actor network and critic network update their parameters using the TD error δ_t . DRLIS continues this process after multiple steps, as an estimate $\hat{A}_t$ of the advantage function A_t , which can be written as:

$$\hat{A}_t = -V_{\pi_\theta}(s_t) + r_t + \gamma r_{t+1} + \dots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_{\pi_\theta}(s_T). \quad (44)$$

DRLIS maintains three networks, one critical network, and two actor networks (i.e., the old actor and the new actor), representing the old policy function $\pi_{\theta_{old}}$ and the new policy function π_θ , as discussed in Section 5.1. Algorithm 1 describes DRLIS for the weighted cost optimization problem in edge and fog computing environments.

Algorithm 1: DRLIS for weighted cost optimization

Input : new actor network Π_θ with parameter θ ; old actor network $\Pi_{\theta_{old}}$ with parameter θ_{old} , where $\theta_{old} = \theta$; critic network V_μ with parameter μ ; max time step T ; update epoch K ; policy objective function coefficient a_c ; value function loss function coefficient a_v ; entropy bonus coefficient a_e ; clipping ratio ϵ

```

1 while True do
2   servers ← GetServers();
3   task ← GetTask();
4   if servers ≠ serversold then
5     agent ← InitializeAgent(servers);
6     serversold ← servers;
7   end if
8   s1 ← GeneralizeState(servers, task);
9   D ← InitializeBuffer();
10  for t ← 1 to T do
11    at ←  $\Pi_\theta(s_t)$ ;
12    Schedule(task, at);
13    rt ← GetReward();
14    servers ← GetServers();
15    if servers ≠ serversold then
16      break;
17    end if
18    task ← GetTask();
19    st+1 ← GeneralizeState(servers, task);
20    ut = (st, at, rt);
21    D.Append(ut);
22  end for
23   $\hat{A}_t \leftarrow -V_\mu(s_t) + r_t + \gamma r_{t+1} + \dots + \gamma^{T-t+1} r_{T-1} + \gamma^{T-t} V_\mu(s_T)$ ;
24  for k ← 1 to K do
25     $L^{CLIP}(\theta) = \sum_1^t \min(\frac{\Pi_\theta(a_t|s_t)}{\Pi_{\theta_{old}}(a_t|s_t)} \hat{A}_t, \text{clip}(\frac{\Pi_\theta(a_t|s_t)}{\Pi_{\theta_{old}}(a_t|s_t)}, 1 - \epsilon, 1 + \epsilon) \hat{A}_t)$ ;
26     $L^{VF}(\mu) = \sum_1^t (V_\mu(s_t) - \hat{A}_t)^2$ ;
27     $L^{ET}(\theta) = \sum_1^t \text{Entropy}(\Pi_\theta(a_t|s_t))$ ;
28     $L(\theta, \mu) = -a_c L^{CLIP}(\theta) + a_v L^{VF}(\mu) - a_e L^{ET}(\theta)$ ;
29    update  $\theta$  and  $\mu$  with  $L(\theta, \mu)$  by Adam optimizer;
30  end for
31   $\theta_{old} \leftarrow \theta$ ;
32 end while

```

We consider a scheduler that is implemented based on DRLIS. When this scheduler receives a scheduling request from an IoT application, it obtains information about the set of servers currently available and initializes a DRL agent based on the information. This agent contains three deep neural networks, a new actor network Π_θ with parameter θ , an old actor network $\Pi_{\theta_{old}}$ with parameter θ_{old} , where $\theta_{old} = \theta$, and a critic network V_μ with parameter μ . After that, the scheduler obtains the information about the currently submitted task and generates the current state s_t based on the information regarding the task and servers. Inputting the state s_t to the new actor network Π_θ will output an action a_t , representing the target server to which the current task is to be assigned. The scheduler then assigns the task to the target server and receives the corresponding reward r_t , which is calculated based on Eqs. (32), (33), (34). The reward r_t is essential for indicating the positive or negative impact of the agent's current scheduling policy on the optimization objectives (e.g., IoT application response time and servers load balancing level). Also, a tuple u_t with three values (s_t, a_t, r_t) will be stored in buffer D . The scheduler repeats the process T times until sufficient information is collected to update the neural networks. When updating the neural networks, the estimate of the advantage function is first computed based on Eq. (44). Then the neural networks

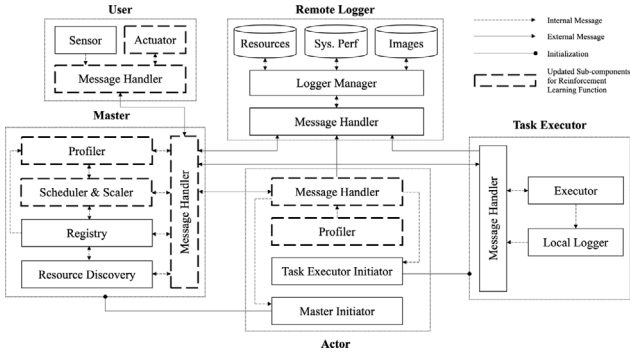


Fig. 3. Updated sub-components for reinforcement learning in FogBus2 framework.

are optimized for K times. Both actor network and critic network use Adam optimizer, and the loss function is computed as:

$$L(\theta, \mu) = -a_c L^{CLIP}(\theta) + a_v L^{VF}(\mu) - a_e L^{ET}(\theta), \quad (45)$$

where $L^{CLIP}(\theta)$ is the policy objective function from Eq. (41), and $L^{VF}(\mu)$ is loss function for the state value function:

$$L^{VF}(\mu) = \sum_{t=1}^T (V_{\mu}(s_t) - \hat{A}_t)^2. \quad (46)$$

And $L^{ET}(\theta)$ is the entropy bonus for the current policy:

$$L^{ET}(\theta) = \sum_{t=1}^T \text{Entropy}(\Pi_{\theta}(a_t | s_t)). \quad (47)$$

In addition, a_c , a_v , and a_e are the coefficients. After updating the neural networks, the parameter θ of the new actor network Π_{θ} will be copied to the old actor network $\Pi_{\theta_{old}}$. Assuming that there are N tasks, from Algorithm 1, the agent will update the policy K times after scheduling T tasks, so the complexity of the algorithm as $O(N + \frac{N}{T}K)$. In practical applications, both T and K as hyperparameters can be customized to suit different computational environments. Thus the computational complexity of the algorithm actually depends on the number of tasks N and can be written as $O(N)$. For the edge/fog environment with limited computational resources, we consider this computational complexity to be acceptable.

5.3. Practical implementation in the FogBus2 framework

We extend the scheduling module of the FogBus2 framework² [14] to design and develop the DRLIS in practice for processing placement requests from different IoT applications in edge and fog computing environments.

FogBus2 is a lightweight container-based distributed/serverless framework (realized using Docker microservices software) for integrating edge and fog/cloud computing environments. A scheduling module is implemented to decide the deployment of heterogeneous IoT applications, enabling the management of distributed resources in the hybrid computing environment. There are five main components within FogBus2 framework, namely *Master*, *Actor*, *RemoteLogger*, *TaskExecutor*, and *User*. Fig. 3 shows the relationship between different components in the FogBus2 framework, and the updated sub-components used to implement the reinforcement learning function.

- *Remote Logger*: It is designed for collecting and storing logs from other components, whether periodic or event-driven.

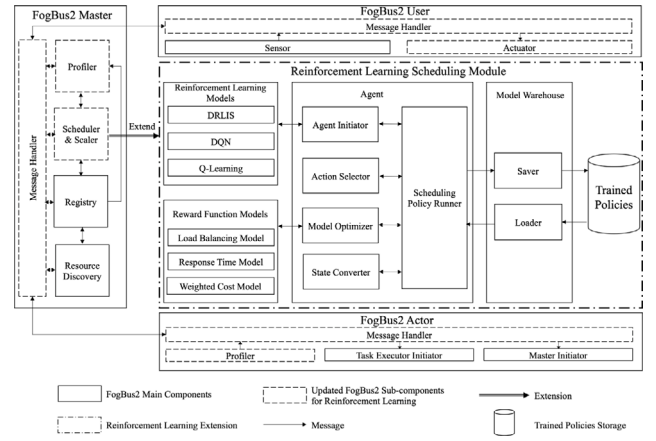


Fig. 4. Reinforcement learning scheduling module in FogBus2 framework.

- *Master*: It contains the scheduling module of FogBus2, responsible for the registration and scheduling of IoT applications. It can also discover resources and self-scale based on the input load. We implement a reinforcement learning scheduling module in the *Scheduler & Scaler* sub-component. Besides, we extend the functionality of the *Profiler* and the *Message Handler* components to allow *Master* components to receive and handle information from other components for reinforcement learning scheduling.
- *Actor*: It informs the *Remote Logger* and *Master* components of the computing resources of the corresponding node to coordinate the resource scheduling of the framework. Furthermore, it is responsible for launching the appropriate *Task Executor* components to process the submitted IoT application. We extend the functionality of the *Profiler* and the *Message Handler* components to allow system characteristics regarding servers to be passed to the reinforcement learning scheduling module in *Master* components.
- *Task Executor*: It is responsible for executing the corresponding tasks of the submitted application. The results are passed to the *Master* component.
- *User*: It runs on IoT devices and is responsible for processing raw data from sensors and users. It sends the processed data to the *Master* component and submits the execution request. We extend the functionality of the *Actuator* and the *Message Handler* components to allow information related to IoT applications to be passed to the reinforcement learning scheduling module in *Master* components.

Fig. 4 shows our implementation of the reinforcement learning scheduling module in the FogBus2 framework. The module can be divided into four sub-modules: (1) Reinforcement Learning Models, (2) Rewards Models, (3) Reinforcement Learning Agent, and (4) Model Warehouse.

- *Reinforcement Learning Models*: This sub-module contains the reinforcement learning models. According to Algorithm 1, we implement a DRLIS-based model. In addition, to evaluate the performance of DRLIS, we also implement DQN and Q-Learning-based models.
- *Rewards Models*: This sub-module contains the models associated with the reward functions. According to Sections 3.2 and 4, we implemented Load Balancing Model, Response Time Model, and Weighted Cost Model. This sub-module is responsible for calculating the reward values based on the information (e.g., CPU and RAM utilization) and transferring them to the *Agent* sub-module.
- *Reinforcement Learning Agent*: This sub-module implements the functions of the reinforcement learning agent. The *Agent Initiator* calls the *Reinforcement Learning Models* sub-module and initializes

² <https://github.com/Cloudslab/FogBus2>.

the corresponding models. The *Action Selector* is responsible for outputting the target server index for the currently scheduled task. The *Model Optimizer* optimizes the running reinforcement learning scheduling policy based on the reward values returned from the *Reward Function Models* sub-module. The *State Converter* is responsible for converting the parameters of the server and IoT application into state vectors that can be recognized by the reinforcement learning scheduling model. The *Scheduling Policy Runner* is the running program of the reinforcement learning scheduling *Agent* and is responsible for receiving submitted tasks, saving or loading the trained policies, and requesting and accessing parameters from other FogBus2 components (e.g., *FogBus2 Actor*, *FogBus2 User*) for the computation of reward functions.

- *Model Warehouse*: This sub-module can save the hyperparameters of the trained scheduling policy to the database and loads the hyperparameters to initialize a well-trained scheduling *Agent*.

Algorithm 2: Reinforcement learning scheduler in FogBus2 framework based on the proposed weighted cost optimization algorithm

Input : master component M ; registered actor component set A ; user component U ; tasks to be processed T

```

1 Scheduler ← InitializeScheduler(DRLIS);
2  $D_A \leftarrow InitializeActorBuffer()$ ;
3  $D_U \leftarrow InitializeUserBuffer()$ ;
4 while True do
5    $U.SubmitTasks(T)$ ;
6    $AvailableActors \leftarrow M.CheckResources(T)$ ;
7   if  $AvailableActors$  is empty then
8      $M.Message(U, Fail)$ ;
9     break;
10  end if
11  foreach  $t_i \in T$  do
12    Scheduler.TaskPlacement( $t_i, A$ );
13     $A.Message(M, I_A)$ ;
14     $D_A.Append(I_A)$ ;
15     $U.Message(M, I_U)$ ;
16     $D_U.Append(I_U)$ ;
17    if UpdateScheduler is True then
18       $Rewards \leftarrow ComputeRewards(D_A, D_U)$ ;
19      Scheduler.Update();
20    end if
21  end foreach
22 end while

```

Algorithm 2 summarizes the scheduling mechanism based on DRLIS. The framework first initializes a scheduler, based on Algorithm 1. In addition, two buffers D_A and D_U for storing information from the *Actor* component and the *User* component are also initialized. After the *User* component submits the IoT application to be processed, the *Master* component first checks whether the *Actor* components that have been registered to the framework have the corresponding resources to process the application. If true, the IoT application which contains one or multiple tasks will be scheduled; otherwise, the *Master* component will inform the *User* component that the current application cannot be processed. For each task of an IoT application, the scheduler will place it to the target *Actor* component for execution based on Algorithm 1. After that, the *Actor* component sends the relevant information (i.e., CPU utilization, RAM utilization, etc.) to the *Master* component, which is stored in the buffer D_A . The *User* component also sends relevant information (i.e., response time, the result of task execution, etc.) to the *Master* component, which is stored in the buffer D_U . When the *Master* collects sufficient information, it will update the scheduler, where the data in D_A and D_U are used to compute the reward for each step, as discussed in Algorithm 1 and Eqs. (32), (33), (34).

6. Performance evaluation

In this section, we first describe the experimental setup and sample applications used in the evaluation. Then, we investigate the hyperparameters of DRLIS. Finally, we discuss the performance of DRLIS by comparing it with its counterparts.

6.1. Experiment setup

We first give a short introduction about the experimental environment and describe the IoT applications used in the experiment. Next, the baseline algorithms used to compare with DRLIS are presented.

6.1.1. Experiment environment

As discussed in Section 5.3, we implemented a scheduler based on DRLIS in the FogBus2 framework, and we use this scheduler for evaluation. We consider a heterogeneous experimental environment consisting of IoT devices, resource-limited fog servers, and resource-rich cloud servers. To simulate the heterogeneous multi-cloud computing environment, we used two instances of Nectar Cloud infrastructure (Intel Xeon 2 cores @2.0 GHz, 9 GB RAM, and Intel Xeon 16 cores @2.0 GHz, 64 GB RAM) and one instance of AWS Cloud (AMD EPYC 2 cores @2.2 GHz, 4GB RAM). In the fog computing environment, to reflect the heterogeneity of the servers, we used a Raspberry Pi 3B (Broadcom BCM2837 4 cores @1.2 GHz, 1 GB RAM), a MacBook Pro (Apple M1 Pro 8 cores, 16 GB RAM), and a Linux virtual machine (Intel Core i5 2 cores @3.1 GHz, 4 GB RAM). In addition, the IoT devices are configured with 2 cores @3.2 GHz and 4 GB RAM. Furthermore, we profiled the average bandwidth (i.e., data rate) and latency between servers as follows: the latency between the IoT device and the cloud server is around 15 ms, and the bandwidth is around 6 MB/s, while the latency between the IoT device and the fog server is around 3 ms, and the bandwidth is around 25 MB/s. Also, both w_1 and w_2 are set to 0.5 in Eq. (19), meaning that the importance of load balancing and response time are equal.

6.1.2. Sample IoT applications

We used four IoT applications for evaluating the performance of the scheduler based on DRLIS. All applications implement both real-time and non-real-time features. Real-time means that the application can receive live streams and non-real-time means that the application can receive pre-recorded video files. Specifically, applications follow a sensor-actuator architecture, with each application operating as a single data stream. Sensors (e.g., cameras) capture environmental information and process it into data patterns (e.g., image frames) that will be forwarded to surrogate servers for processing, while actuators receive the processed data and represent the final outcome to the user. In addition, all applications provide a parameter called *application label*, which can be used to set the frame size in the video. These applications are described as follows:

- *Face Detection* [15]: Detects and captures human faces. The human faces in the video are marked by squares. This application is implemented based on OpenCV.³
- *Color Tracking* [15]: Tracks colors from video. The user can dynamically configure the target colors through the GUI provided by the application. This application is implemented based on OpenCV.³
- *Face And Eye Detection* [15]: In addition to detecting and capturing human faces, the application also detects and captures human eyes. This application is implemented based on OpenCV.³
- *Video OCR* [14]: Recognizes and extracts text information from the video and transmits it back to the user. The application will automatically filter out keyframes. This application is implemented based on Google's Tesseract-OCR Engine.⁴

³ <https://github.com/opencv/opencv>.

⁴ <https://github.com/tesseract-ocr/tesseract>.

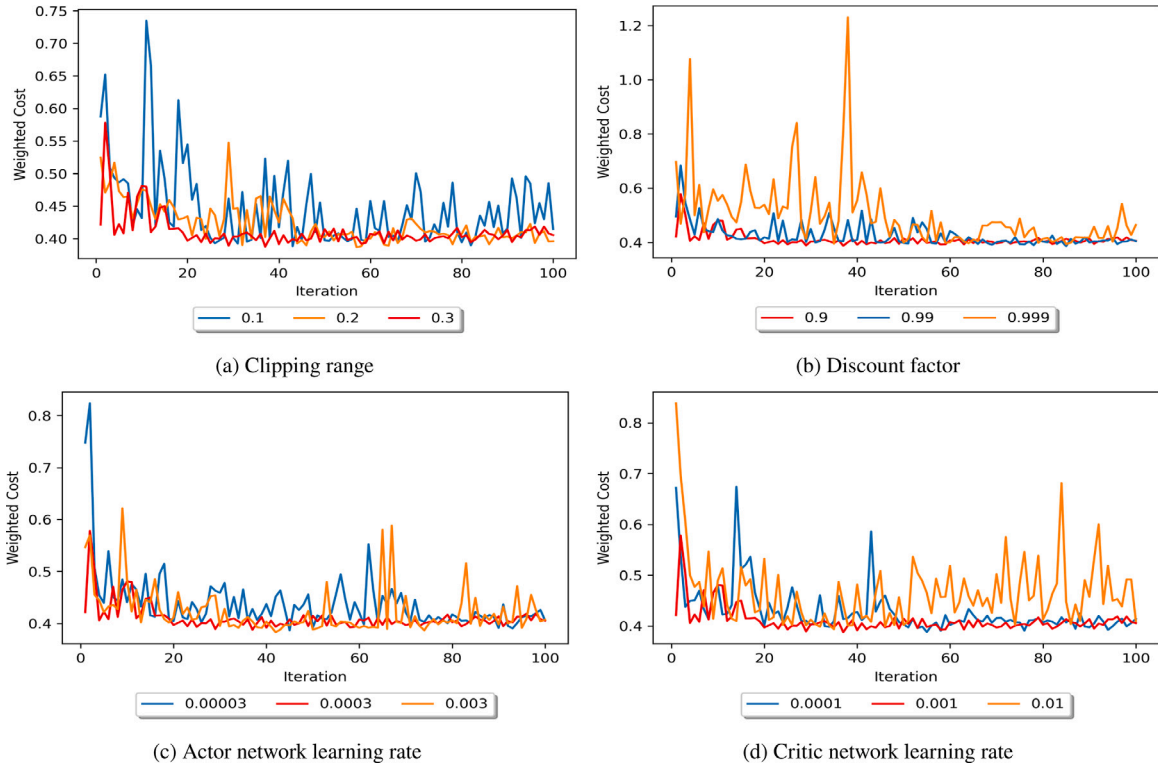


Fig. 5. Hyperparameters tuning results.

6.1.3. Baseline algorithms

To evaluate the performance of DRLIS, three other schedulers based on metaheuristic algorithms and reinforcement learning techniques are implemented, as follows:

- **DQN**: It is one of the most adapted techniques in deep reinforcement learning, which constructs an end-to-end architecture from perception to decision. This algorithm has been used by many works in the current literature such as [26–28], and [29]. To compare with our proposed algorithm, we implement a DQN-based scheduler and integrate it into the FogBus2 framework. This scheduler can minimize the weighted load balancing and response time cost.
- **Q-Learning**: This technique belongs to value-based reinforcement learning techniques that combine the Monte Carlo method and the TD method. Its ultimate goal is to learn a table (*Q-Table*). Works including [25,55] adopt this technique. To integrate it into the FogBus2 framework, we implemented a scheduling policy. Furthermore, as a comparison, the scheduler can be used in the weighted cost problem to minimize the weighted load balancing and response time cost.
- **NSGA2**: It is a weighted cost genetic algorithm. It adopts the strategy of fast non-dominated sorting and crowding distance to reduce the complexity of the non-dominated sorting genetic algorithm. The algorithm has high efficiency and fast convergence rate [56]. This algorithm is implemented using Pymoo [57].
- **NSGA3**: The framework of NSGA3 is basically the same as NSGA2, using fast non-dominated sorting to classify population individuals into different non-dominated fronts, and the difference mainly lies in the change of selection mechanism. Compared with NSGA2 using crowding distance to select individuals of the same non-dominated level, NSGA3 introduces well-distributed reference points to maintain population diversity under high-dimensional goals [58]. This algorithm is implemented using Pymoo [57].

6.2. Hyperparameter tuning

The scheduler based on DRLIS is implemented via PyTorch. Considering the limited computational resources of some devices in the fog computing environment, both actor network and critic network consist of an input layer, a hidden layer, and an output layer. Henderson et al. [59] investigate the effect of hyperparameter settings on the performance of reinforcement learning models. They survey the literature on different reinforcement learning techniques, list the hyperparameter settings used in the literature, and compare the actual performance of the models under different hyperparameter settings. They compare the performance of the PPO algorithm under different network architectures and the result shows that the model performs best under the network architecture where the hidden layer contains 64 hidden units and the hyperbolic tangent (TanH) function is used as the activation function. Therefore, we used the same network architecture for our experiments. In addition, we performed a grid search to tune the four main hyperparameters (i.e., clipping range, discount factor, learning rate for actor network, and learning rate for critic network), and the results are shown in Fig. 5. The load balancing model control parameters a_1 and a_2 are both set to 0.5 to show the equal importance of CPU and RAM, however, these values can be tuned by users based on the objectives.

All the experiments regarding hyperparameters tuning are conducted in order to solve the weighted cost problem, as discussed in Section 3.2.3. We describe the process of hyperparameters tuning of our reinforcement learning model. For tuning the clipping range ϵ , we followed Schulman et al. [13], who proposed PPO and described that the model performs best with settings of clipping range ϵ among 0.1, 0.2, and 0.3. Fig. 5(a) shows that our model performs best when the clipping range ϵ is set to 0.3. For the discount factor γ , we reviewed related work on DRL in order to understand the common range for γ . According to [13,60], the best setting for γ sits somewhere among {0.9–0.999}. Accordingly, to keep the search area for tuning γ in a viable range, we used the nominated values in these works and found that our model converges faster when γ is set to 0.9. Fig. 5(b) shows

Table 3

The hyperparameters setting for DRLIS.

DRLIS hyperparameter	Value
Neural network layers	3
Hidden layer units	64
Optimization method	Adam
Activation function	TanH
Clipping range ϵ	0.3
Discount factor γ	0.9
Actor learning rate lr_a	0.0003
Critic learning rate lr_c	0.001
Policy objective function coefficient a_c	1
Value function loss function coefficient a_v	0.5
Entropy bonus coefficient a_e	0.01
Load balancing model CPU control parameter a_1	0.5
Load balancing model RAM control parameter a_2	0.5

Table 4

The hyperparameters setting for baseline techniques.

DQN hyperparameter	Value
Neural network layers	3
Hidden layer units	64
Optimization method	Adam
Activation function	ReLU
Discount factor	0.99
Learning rate	0.0001
Exploration rate	1
Exploration decay	0.9
Minimum exploration	0.05
Q-Learning hyperparameter	Value
Discount factor	0.9
Learning rate	0.1
NSGA2 and NSGA3 hyperparameter	Value
Population size	200
Generation numbers	100

the tuning process of γ . Based on the similar approach for tuning ϵ and γ , for tuning the actor network learning rate lr_a , we referred to [13,59,61] for designing our tuning range. Accordingly, we used 0.003, 0.0003, and 0.00003 to tune lr_a . Fig. 5(c) shows that our model performs best when the lr_a is set to 0.0003. Considering the same approach for tuning, we followed [62–64] and set our tuning range among {0.01, 0.001, 0.0001} and found that our model works best when lr_c is 0.001. Fig. 5(d) shows the performance of our model under different settings for lr_c . Overall, the deep neural network and training hyperparameters setting is presented in Table 3. Besides, we also tune the hyperparameters for baseline techniques to fairly study their performance. The corresponding results are shown in Table 4.

6.3. Performance study

We performed two experiments to evaluate DRLIS compared to its counterparts, regarding the load balancing of the servers, the response time of the IoT applications, and the weighted cost.

6.3.1. Cost vs. policy update analysis

In this experiment, we investigate the algorithm performance in different iterations when the policy is updated. We used the four applications mentioned in Section 6.1.2 for training with the resolution parameter set to 480, and the maximum number of iterations is set to 100. The training results of algorithms with the three optimization objectives are shown in Fig. 6.

As shown in Fig. 6(a), when optimizing the load-balancing problem of the servers, the average computational resource variance of the servers is lower for the Q-Learning-based, DQN-based, and DRLIS-based

schedulers than for the NSGA2-based and NSGA3-based schedulers. Moreover, only the reinforcement learning-based scheduler can achieve a stable convergence state. However, the Q-Learning-based scheduler requires more than 60 iterations before reaching a converged state, and the DQN-based scheduler requires more than 80 iterations, while the DRLIS-based scheduler only requires about 20 updates to converge to a similar stable state. The NSGA2-based and NSGA3-based schedulers are unable to reach the convergence state. As shown in Fig. 6(b), when optimizing the response time problem of the application, unlike the former problem, all schedulers can converge. However, the average response time of Q-Learning-based, DQN-based, and DRLIS-based schedulers is still lower than that of NSGA2-based and NSGA3-based schedulers. In addition, the DRLIS-based scheduler still outperforms the Q-Learning-based and the DQN-based schedulers in terms of convergence speed. Finally, as Fig. 6(c) shows, when optimizing the weighted cost problem, similar to the load balancing problem, the average cost is lower for the Q-Learning-based, DQN-based, and DRLIS-based schedulers than for the NSGA2 and NSGA3-based schedulers, and only the first three can reach a stable convergence state. Moreover, although the Q-Learning-based, DQN-based, and DRLIS-based schedulers have similar final convergence levels, the DRLIS-based scheduler converges much faster than the Q-Learning-based and DQN-based schedulers. This proves that the DRLIS-based scheduler outperforms the other techniques in terms of average cost, convergence, and convergence speed during the training phase.

In the evaluation phase, we set the resolution to 240, which will make the demand for computational resources and response time of the IoT application different from the training phase. The evaluation phase results of the different algorithms regarding the three optimization objectives are shown in Fig. 7. It can be observed that when the optimization objective is server load balancing, IoT application response time, and weighted cost, respectively, the schedulers based on different algorithms have similar performances as the training phase. Specifically, only the cost of the Q-Learning-based, DQN-based, and DRLIS-based schedulers converges, and the cost of the NSGA2-based and NSGA3-based schedulers fluctuates up and down in a higher range. Moreover, the average and final costs of the Q-Learning-based, DQN-based, and DRLIS-based schedulers are significantly lower than those of the NSGA2-based and NSGA3-based schedulers during the evaluation phase. In addition, in the weighted cost scenario, the DRLIS-based scheduler can converge the cost to a stable level after about 30 policy updates, while the Q-Learning-based scheduler usually takes about 60 updates to converge to a slightly higher level, and the DQN-based scheduler needs more than 80 updates to converge to the same level. Overall, compared with the Q-Learning-based scheduler, which can converge stably and with the fastest convergence speed in the baseline algorithms, the average performance of the DRLIS-based scheduler improves by 55%, 37%, and 50%, in terms of servers load balancing, IoT application response time, and weighted cost, respectively.

6.3.2. Scheduling overhead analysis

In this section, we investigate the scheduling overhead of different techniques-based schedulers when handling IoT applications. The environment settings are the same as Section 6.1.1, and the resolution of the IoT applications is set to 480. For each scheduler, we repeat the experiment for 100 rounds, feeding four IoT applications to the scheduler in each round. Besides, we define the average scheduling overhead as $T_{ave} = \frac{T_{total}}{100}$, where T_{total} represents the total overhead spent by the scheduler to handle the applications in 100 rounds.

Fig. 8 depicts the average scheduling overhead T_{ave} with a 95% Confidence Interval (CNFI) of schedulers based on different technologies when handling IoT applications. It is obvious that the scheduling overheads of reinforcement learning techniques (i.e., DRLIS, DQN, Q-Learning) are usually lower than metaheuristics techniques (i.e., NSGA2, NSGA3). In addition, the 95% CNFI of the scheduling overhead

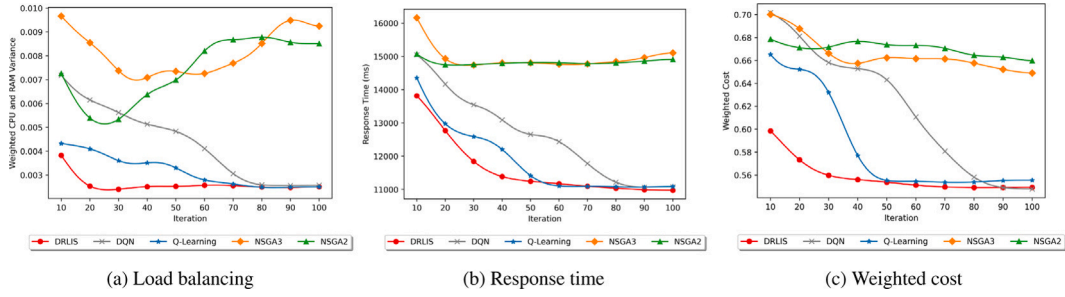


Fig. 6. Cost vs. policy update analysis — train phase.

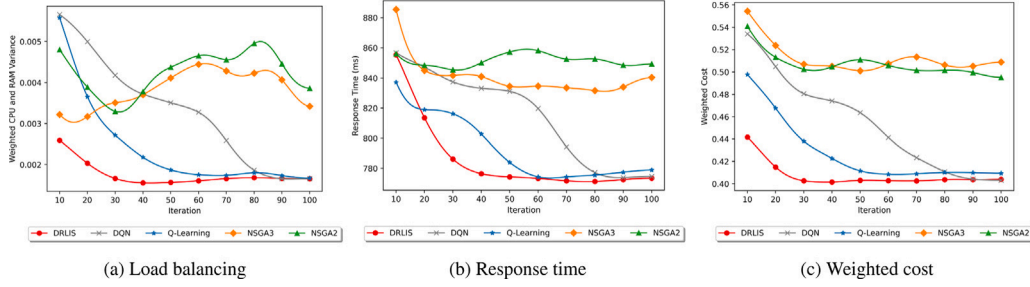


Fig. 7. Cost vs. policy update analysis — evaluation phase.

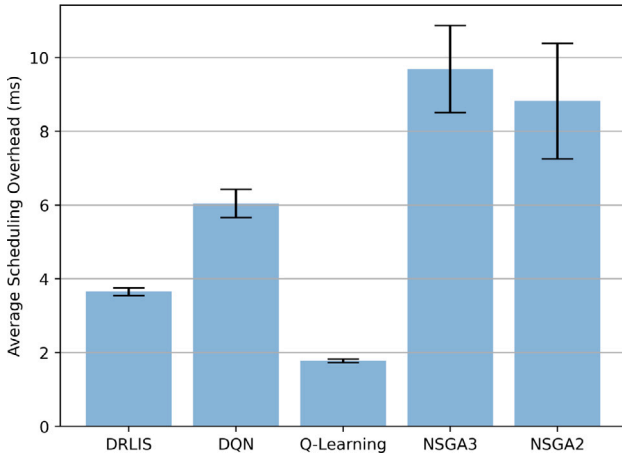


Fig. 8. Average scheduling overhead with a 95% CNFI.

of reinforcement learning techniques is also much shorter than meta-heuristic techniques. Specifically, the scheduling overhead of DRLIS is more than 50% lower than NSGA2 and NSGA3, and more than 33% lower than DQN, but it is about 2 ms more than Q-Learning. However, considering that the convergence speed of DRLIS is much faster than that of Q-Learning, as discussed in Section 6.3.1, the increased overhead cost of DRLIS over Q-Learning can be negligible. Therefore, in the heterogeneous edge and fog computing environment, our proposed DRLIS-based algorithm can handle the weighted cost optimization problem of IoT applications more efficiently than other techniques.

7. Conclusions and future work

In this paper, we proposed DRLIS, a DRL-based algorithm to solve the weighted cost optimization problem for IoT applications scheduling in heterogeneous edge and fog computing environments. First, we proposed corresponding cost models for optimizing load balancing

and response time in heterogeneous edge and fog computing environments and formulate a weighted cost model based on both of them. In addition, we implemented a practical scheduler in the Fog-Bus2 function-as-a-service framework for scheduling IoT applications. Compared to existing work, DRLIS has significant advantages in convergence speed, optimization cost, and scheduling overhead. Through extensive experiments and comparisons with other works in the literature, DRLIS achieves performance improvements of up to 49%, 60%, and 55% in terms of load balancing, response time, and weighted cost, respectively.

For future work, considering the limited resources and the distribution of the devices in edge computing, we plan to explore distributed deep reinforcement learning to further improve the scheduler's performance. Also, we plan to consider more models to extend our proposed weighted cost model, including economic aspects and energy consumption aspects in large-scale serverless computing environments. In addition, to optimize the performance of IoT applications involving GPU tasks (e.g., image processing oriented applications), we will extend Fog-Bus2 framework to consider resource usage when scheduling such applications on Application-Specific Integrated Circuit (ASIC)/GPU-based edge and cloud servers for more efficient performance.

CRedit authorship contribution statement

Zhiyu Wang: Problem formulation, Implementation of the work, Writing the paper. **Mohammad Goudarzi:** Problem formulation, Design of the DRL-based framework, Consultation for implementation of DRL, Editing the paper. **Mingming Gong:** Supervisor, Providing technical advises. **Rajkumar Buyya:** Supervisor, Providing technical advises.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

References

- [1] G.S.S. Chalapathi, V. Chamola, A. Vaish, R. Buyya, Industrial internet of things (IIoT) applications of edge and fog computing: A review and future directions, in: *Fog/Edge Computing for Security, Privacy, and Applications*, Springer, 2021, pp. 293–325.
- [2] S. Azizi, M. Shojafar, J. Abawajy, R. Buyya, Deadline-aware and energy-efficient IIoT task scheduling in fog computing systems: A semi-greedy approach, *J. Netw. Comput. Appl.* 201 (2022) 103333.
- [3] A.J. Ferrer, J.M. Marqués, J. Jorba, Towards the decentralised cloud: Survey on approaches and challenges for mobile, ad hoc, and edge computing, *ACM Comput. Surv.* 51 (6) (2019) 1–36.
- [4] M. Goudarzi, H. Wu, M. Palaniswami, R. Buyya, An application placement technique for concurrent IIoT applications in edge and fog computing environments, *IEEE Trans. Mob. Comput.* 20 (4) (2020) 1298–1311.
- [5] L. Catarinucci, D. de Donno, L. Mainetti, L. Palano, L. Patrono, M.L. Stefanizzi, L. Tarricone, An IIoT-aware architecture for smart healthcare systems, *IEEE Internet Things J.* 2 (6) (2015) 515–526.
- [6] L. Liu, S. Lu, R. Zhong, B. Wu, Y. Yao, Q. Zhang, W. Shi, Computing systems for autonomous driving: State of the art and challenges, *IEEE Internet Things J.* 8 (8) (2021) 6469–6486.
- [7] M. Goudarzi, M.S. Palaniswami, R. Buyya, A distributed deep reinforcement learning technique for application placement in edge and fog computing environments, *IEEE Trans. Mob. Comput.* 22 (5) (2023) 2491–2505.
- [8] A. Brogi, S. Forti, QoS-aware deployment of IIoT applications through the fog, *IEEE Internet Things J.* 4 (5) (2017) 1185–1192.
- [9] M. Goudarzi, M. Palaniswami, R. Buyya, Scheduling IIoT applications in edge and fog computing environments: a taxonomy and future directions, *ACM Comput. Surv.* 55 (7) (2022) 1–41.
- [10] X. Ma, H. Gao, H. Xu, M. Bian, An IIoT-based task scheduling optimization scheme considering the deadline and cost-aware scientific workflow for cloud computing, *EURASIP J. Wireless Commun. Networking* 2019 (1) (2019) 1–19.
- [11] Z. Wang, M. Goudarzi, J. Aryal, R. Buyya, Container orchestration in edge and fog computing environments for real-time IIoT applications, in: *Proceedings of the Computational Intelligence and Data Analytics (ICCIDA)*, Springer, 2022, pp. 1–21.
- [12] E. Li, Z. Zhou, X. Chen, Edge intelligence: On-demand deep learning model co-inference with device-edge synergy, in: *Proceedings of the 2018 Workshop on Mobile Edge Communications*, 2018, pp. 31–36.
- [13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, O. Klimov, Proximal policy optimization algorithms, 2017, arXiv preprint [arXiv:1707.06347](https://arxiv.org/abs/1707.06347).
- [14] Q. Deng, M. Goudarzi, R. Buyya, FogBus2: a lightweight and distributed container-based framework for integration of IIoT-enabled systems with edge and cloud computing, in: *Proceedings of the International Workshop on Big Data in Emergent Distributed Environments*, 2021, pp. 1–8.
- [15] M. Goudarzi, Q. Deng, R. Buyya, Resource management in edge and fog computing using FogBus2 framework, 2021, arXiv preprint [arXiv:2108.00591](https://arxiv.org/abs/2108.00591).
- [16] K. Deb, S. Agrawal, A. Pratap, T. Meyarivan, A fast elitist non-dominated sorting genetic algorithm for multi-objective optimization: NSGA-II, in: *Proceedings of the International Conference on Parallel Problem Solving from Nature*, Springer, 2000, pp. 849–858.
- [17] K. Deb, H. Jain, An evolutionary many-objective optimization algorithm using reference-point-based nondominated sorting approach, part I: solving problems with box constraints, *IEEE Trans. Evol. Comput.* 18 (4) (2013) 577–601.
- [18] C.J. Watkins, P. Dayan, Q-learning, *Mach. Learn.* 8 (1992) 279–292.
- [19] J. Liu, Y. Mao, J. Zhang, K.B. Letaief, Delay-optimal computation task scheduling for mobile-edge computing systems, in: *Proceedings of the IEEE International Symposium on Information Theory (ISIT)*, 2016, pp. 1451–1455.
- [20] C.-g. Wu, W. Li, L. Wang, A.Y. Zomaya, Hybrid evolutionary scheduling for energy-efficient fog-enhanced internet of things, *IEEE Trans. Cloud Comput.* 9 (2) (2021) 641–653.
- [21] Y. Sun, F. Lin, H. Xu, Multi-objective optimization of resource scheduling in fog computing using an improved NSGA-II, *Wirel. Pers. Commun.* 102 (2) (2018) 1369–1385.
- [22] F. Hoseiny, S. Azizi, M. Shojafar, F. Ahmadiazar, R. Tafazolli, PGA: A priority-aware genetic algorithm for task scheduling in heterogeneous fog-cloud computing, in: *Proceedings of the IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, 2021, pp. 1–6.
- [23] I.M. Ali, K.M. Sallam, N. Moustafa, R. Chakraborty, M. Ryan, K.-K.R. Choo, An automated task scheduling model using non-dominated sorting genetic algorithm II for fog-cloud systems, *IEEE Trans. Cloud Comput.* 10 (4) (2022) 2294–2308.
- [24] F. Ramezani Shahidani, A. Ghasemi, A. Toroghi Haghghat, A. Keshavarzi, Task scheduling in edge-fog-cloud architecture: a multi-objective load balancing approach using reinforcement learning algorithm, *Computing* (2023) 1–23.
- [25] J.-y. Baek, G. Kaddoum, S. Garg, K. Kaur, V. Gravel, Managing fog networks using reinforcement learning based load balancing algorithm, in: *Proceedings of the IEEE Wireless Communications and Networking Conference (WCNC)*, IEEE, 2019, pp. 1–7.
- [26] X. Jie, T. Liu, H. Gao, C. Cao, P. Wang, W. Tong, A DQN-based approach for online service placement in mobile edge computing, in: *Proceedings of the 16th EAI International Conference on Collaborative Computing: Networking, Applications and Worksharing*, Springer, 2021, pp. 169–183.
- [27] X. Xiong, K. Zheng, L. Lei, L. Hou, Resource allocation based on deep reinforcement learning in IIoT edge computing, *IEEE J. Sel. Areas Commun.* 38 (6) (2020) 1133–1146.
- [28] J. Wang, L. Zhao, J. Liu, N. Kato, Smart resource allocation for mobile edge computing: A deep reinforcement learning approach, *IEEE Trans. Emerg. Top. Comput.* 9 (3) (2021) 1529–1541.
- [29] L. Huang, X. Feng, C. Zhang, L. Qian, Y. Wu, Deep reinforcement learning-based joint task offloading and bandwidth allocation for multi-user mobile edge computing, *Digit. Commun. Netw.* 5 (1) (2019) 10–17.
- [30] X. Chen, H. Zhang, C. Wu, S. Mao, Y. Ji, M. Bennis, Optimized computation offloading performance in virtual edge computing systems via deep reinforcement learning, *IEEE Internet Things J.* 6 (3) (2019) 4005–4018.
- [31] Y. Zheng, H. Zhou, R. Chen, K. Jiang, Y. Cao, SAC-based computation offloading and resource allocation in vehicular edge computing, in: *Proceedings of the IEEE INFOCOM - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, 2022, pp. 1–6.
- [32] T. Zhao, F. Li, L. He, Secure video offloading in MEC-enabled IIoT networks: A multi-cell federated deep reinforcement learning approach, *IEEE Trans. Ind. Inform.* (2023) 1–12.
- [33] L. Liao, Y. Lai, F. Yang, W. Zeng, Online computation offloading with double reinforcement learning algorithm in mobile edge computing, *J. Parallel Distrib. Comput.* 171 (2023) 28–39.
- [34] V. Sethi, S. Pal, FedDOVE: A Federated Deep Q-learning-based Offloading for Vehicular fog computing, *Future Gener. Comput. Syst.* 141 (2023) 96–105.
- [35] P. Li, W. Xie, Y. Yuan, C. Chen, S. Wan, Deep reinforcement learning for load balancing of edge servers in IIoT, *Mob. Netw. Appl.* 27 (4) (2022) 1461–1474.
- [36] X. Chu, M. Zhu, H. Mao, Y. Qiu, Task offloading for multi-gateway-assisted mobile edge computing based on deep reinforcement learning, in: *Proceedings of the IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, 2022, pp. 3234–3241.
- [37] F. Xue, Q. Hai, T. Dong, Z. Cui, Y. Gong, A deep reinforcement learning based hybrid algorithm for efficient resource scheduling in edge computing environment, *Inform. Sci.* 608 (2022) 362–374.
- [38] S. Pallevatta, V. Kostakos, R. Buyya, Placement of microservices-based IIoT applications in fog computing: A taxonomy and future directions, *ACM Comput. Surv.* 55 (2023) 1–43.
- [39] W. Zhu, M. Goudarzi, R. Buyya, Flight: A lightweight federated learning framework in edge and fog computing, 2023, arXiv preprint [arXiv:2308.02834](https://arxiv.org/abs/2308.02834).
- [40] X. Qiu, W. Zhang, W. Chen, Z. Zheng, Distributed and collective deep reinforcement learning for computation offloading: A practical perspective, *IEEE Trans. Parallel Distrib. Syst.* 32 (5) (2020) 1085–1101.
- [41] N.H. Tran, W. Bao, A. Zomaya, M.N. Nguyen, C.S. Hong, Federated learning over wireless networks: Optimization model design and analysis, in: *Proceedings of the IEEE INFOCOM*, IEEE, 2019, pp. 1387–1395.
- [42] J. Ji, K. Zhu, L. Cai, Trajectory and communication design for cache-enabled UAVs in cellular networks: A deep reinforcement learning approach, *IEEE Trans. Mob. Comput.* (2022).
- [43] S. Fujimoto, H. van Hoof, D. Meger, Addressing function approximation error in actor-critic methods, in: J. Dy, A. Krause (Eds.), *Proceedings of the 35th International Conference on Machine Learning*, in: *Proceedings of Machine Learning Research*, 80, PMLR, 2018, pp. 1587–1596.
- [44] T. Haarnoja, A. Zhou, P. Abbeel, S. Levine, Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor, in: J. Dy, A. Krause (Eds.), *Proceedings of the 35th International Conference on Machine Learning*, in: *Proceedings of Machine Learning Research*, 80, PMLR, 2018, pp. 1861–1870.
- [45] S. Mysore, B.E. Mabsout, R. Mancuso, K. Saenko, Honey. I shrunk the actor: A case study on preserving performance with smaller actors in actor-critic RL, in: *2021 IEEE Conference on Games (CoG)*, 2021, pp. 01–08.
- [46] P. Christodoulou, Soft actor-critic for discrete action settings, 2019, arXiv preprint [arXiv:1910.07207](https://arxiv.org/abs/1910.07207).
- [47] H. Wang, Y. Ye, J. Zhang, B. Xu, A comparative study of 13 deep reinforcement learning based energy management methods for a hybrid electric vehicle, *Energy* 266 (2023) 126497.
- [48] J. Zhu, F. Wu, J. Zhao, An overview of the action space for deep reinforcement learning, in: *Proceedings of the 4th International Conference on Algorithms, Computing and Artificial Intelligence*, 2021, pp. 1–10.
- [49] R.S. Sutton, D. McAllester, S. Singh, Y. Mansour, Policy gradient methods for reinforcement learning with function approximation, *Adv. Neural Inf. Process. Syst.* 12 (1999).
- [50] R. Huang, T. Yu, Z. Ding, S. Zhang, Policy gradient, in: *Deep Reinforcement Learning: Fundamentals, Research and Applications*, Springer, 2020, pp. 161–212.
- [51] J. Schulman, S. Levine, P. Abbeel, M. Jordan, P. Moritz, Trust region policy optimization, in: *Proceedings of the International Conference on Machine Learning*, PMLR, 2015, pp. 1889–1897.

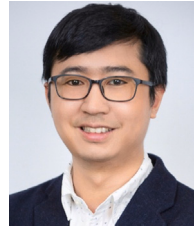
- [52] T. Van Erven, P. Harremos, Rényi divergence and Kullback-Leibler divergence, *IEEE Trans. Inform. Theory* 60 (7) (2014) 3797–3820.
- [53] S. Shao, W. Luk, Customised pearlmuter propagation: A hardware architecture for trust region policy optimisation, in: *Proceedings of the 27th International Conference on Field Programmable Logic and Applications (FPL)*, IEEE, 2017, pp. 1–6.
- [54] S. Li, R. Wang, M. Tang, C. Zhang, Hierarchical reinforcement learning with advantage-based auxiliary rewards, *Adv. Neural Inf. Process. Syst.* 32 (2019).
- [55] S. Aljanabi, A. Chalechale, Improving IoT services using a hybrid fog-cloud offloading, *IEEE Access* 9 (2021) 13775–13788.
- [56] L. Yliniemi, K. Tumer, Multi-objective multiagent credit assignment in reinforcement learning and nsga-ii, *Soft Comput.* 20 (10) (2016) 3869–3887.
- [57] J. Blank, K. Deb, Pymoo: Multi-objective optimization in python, *IEEE Access* 8 (2020) 89497–89509.
- [58] X. Li, X. Li, K. Wang, S. Yang, Y. Li, Achievement scalarizing function sorting for strength Pareto evolutionary algorithm in many-objective optimization, *Neural Comput. Appl.* 33 (2021) 6369–6388.
- [59] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, D. Meger, Deep reinforcement learning that matters, in: *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 32, 2018.
- [60] D. Wei, N. Xi, X. Ma, M. Shojafar, S. Kumari, J. Ma, Personalized privacy-aware task offloading for edge-cloud-assisted industrial internet of things in automated manufacturing, *IEEE Trans. Ind. Inform.* 18 (11) (2022) 7935–7945.
- [61] N. Bjorck, C.P. Gomes, B. Selman, K.Q. Weinberger, Understanding batch normalization, *Adv. Neural Inf. Process. Syst.* 31 (2018).
- [62] C. Huang, R. Mo, C. Yuen, Reconfigurable intelligent surface assisted multiuser MISO systems exploiting deep reinforcement learning, *IEEE J. Sel. Areas Commun.* 38 (8) (2020) 1839–1850.
- [63] F. Fu, Y. Kang, Z. Zhang, F.R. Yu, Transcoding for live streaming-based on vehicular fog computing: An actor-critic DRL approach, in: *Proceedings of the IEEE INFOCOM - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, 2020, pp. 1015–1020.
- [64] R. Islam, P. Henderson, M. Gomrokchi, D. Precup, Reproducibility of benchmarked deep reinforcement learning tasks for continuous control, 2017, arXiv preprint [arXiv:1708.04133](https://arxiv.org/abs/1708.04133).



Zhiyu Wang is working towards the Ph.D. degree at the Cloud Computing and Distributed Systems (CLOUDS) Laboratory, Department of Computing and Information Systems, the University of Melbourne. His research interests include Edge/Fog Computing, Internet of Things (IoT), Distributed Systems, and Artificial Intelligence. He is currently working on leveraging AI techniques to enhance resource management in edge/fog and cloud computing environments.



Mohammad Goudarzi is working as a senior research associate at the University of New South Wales (UNSW), Sydney, Australia. He completed his Ph.D. at the Department of Computing and Information Systems, the University of Melbourne, Australia. His research interests include Internet of Things (IoT), Cloud/Fog/Edge Computing, Cyber Security, Distributed Systems, and Machine Learning. He received Oracle's Cloud Architect of the Year Award 2022, IEEE TCCLD Outstanding Ph.D. Thesis Award 2022, IEEE TCSC Outstanding Ph.D. Dissertation Award 2022, and IEEE Outstanding Service Award 2021. Media outlets such as Australian Financial Review (AFR) and Oracle Blogs have covered his research findings.



Mingming Gong is currently a Senior Lecturer in data science with the School of Mathematics and Statistics, The University of Melbourne, Melbourne, VIC, Australia. He has authored/coauthored more than 30 research papers on top venues, such as International Conference on Machine Learning (ICML), Neural Information Processing Systems (NeurIPS), the IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS, and the IEEE TRANSACTIONS ON IMAGE PROCESSING, with more than ten oral/spotlight presentations. His research interests include causal reasoning, machine learning, and computer vision. Dr. Gong received the Discovery Early Career Researcher Award from the Australian Research Council in 2020.



Dr. Rajkumar Buyya is a Redmond Barry Distinguished Professor and Director of the Cloud Computing and Distributed Systems (CLOUDS) Laboratory at the University of Melbourne, Australia. He has authored over 625 publications and seven text books including "Mastering Cloud Computing" published by McGraw Hill, China Machine Press, and Morgan Kaufmann for Indian, Chinese and international markets respectively. He is one of the highly cited authors in computer science and software engineering worldwide (h-index=160, g-index=322, 137900+ citations).